

Software Architecture Patterns

Chapter 1: Microservices Architecture

Microservices architecture structures an application as a collection of services that are highly maintainable, testable, and loosely coupled. Each service is self-contained, independently deployable, and organized around business capabilities.

Chapter 2: Event-Driven Architecture

Event-driven architecture (EDA) is a software design pattern in which decoupled applications can asynchronously publish and subscribe to events via an event broker. This pattern enables loose coupling between services and improves scalability.

Chapter 3: CQRS Pattern

Command Query Responsibility Segregation (CQRS) is a pattern that separates read and write operations for a data store. The write model (commands) uses a normalized relational database, while the read model (queries) uses a denormalized view optimized for specific query patterns.

Chapter 4: Database per Service

Each microservice should own its data. This pattern ensures loose coupling between services by giving each service its own database. Services communicate through APIs rather than sharing databases. This prevents tight coupling at the data layer.

Chapter 5: API Gateway Pattern

An API Gateway provides a single entry point for all client requests. It handles cross-cutting concerns like authentication, rate limiting, SSL termination, and request routing. Popular implementations include Kong, AWS API Gateway, and Azure API Management.

Key Technologies: Python, FastAPI, SQLAlchemy, PostgreSQL, Redis, Docker, Kubernetes

Performance benchmarks show FastAPI achieves 2-3x higher throughput than Flask for async workloads.